

Assignment 1

Shashank Satish Adsule - DA25M005
14 ebruary 2026

1. What was the most difficult part of managing data versions manually?

→ The most challenging aspect of manual data versioning was maintaining **consistency, traceability, and reproducibility** across the pipeline. The difficulty was not technical complexity but the heavy reliance on human discipline.

Key Challenges

1. Manual Lineage Tracking

All preprocessing steps had to be recorded manually in manifest.txt. This created risks because:

- Missing even one entry breaks reproducibility.
- There was no automatic enforcement or validation.
- Re-running preprocessing during debugging required manual manifest updates.

This created high cognitive overhead and increased the risk of silent errors.

2. Fragile File Naming Conventions

Data versions were tracked using names like v1_train_processed.csv. This approach is unreliable because:

- Naming conventions depend on consistent human behavior.
- Multiple similar files can easily cause confusion.
- There is no guarantee that file content actually matches its version label.

3. Configuration Synchronization

Reproducibility depended on keeping three components synchronized:

- config.yaml
- Processed data files
- Model metadata files

If any one component became inconsistent, the entire pipeline produced incorrect results. During testing, I encountered scenarios where training unintentionally used outdated data due to version mismatch.

4. Storage and Artifact Management

Multiple dataset versions and preprocessing artifacts accumulated quickly, and there was no automated method to determine:

- Which artifacts were active
- Which versions were safe to delete
- Which dataset versions were linked to specific models

Conclusion

Manual data versioning significantly increased mental workload and introduced reproducibility risks. Automated data versioning tools would eliminate these problems by tracking lineage using content hashing and pipeline dependency graphs.

2. How did you ensure that the model in production was the same one you evaluated during training?

→ I implemented multiple verification layers to ensure the deployed model matched the trained model. However, this approach relied on detection rather than prevention.

Implemented Safeguards

Metadata Tracking

After training, each model stored metadata including:

- Training timestamp
- Data version
- Git commit reference
- Evaluation metrics

This allowed traceability between the model and its training configuration.

Deployment Logging

API deployments were recorded in `deployment_log.csv` to track when models were deployed and which version was used.

API Model Inspection

A `/model-info` endpoint allowed runtime verification of the model currently loaded in memory.

Manual Deployment Checklist

Before deployment, I manually verified configuration files, model availability, metadata consistency, and active API processes.

Observed Limitations

The largest issue occurred when retraining overwrote an existing model file while the API continued serving the old model from memory. This caused deployment logs and actual runtime behavior to diverge.

Additional problems included:

- No record of stopped deployments
- No checksum validation for model files
- Accidental multiple API instances
- Difficulty tracking which model generated predictions

Conclusion

While manual verification allowed model tracking, it could not prevent incorrect deployments. Automated model registries solve this by enforcing version immutability, checksum validation, and structured deployment workflows.

3. How did you ensure that train.py always used the correct version of the data specified in config.yaml?

→ Correct dataset selection was enforced through configuration-driven path generation and fail-fast validation.

Implementation Approach

Single Configuration Source

All dataset version information was stored in config.yaml, eliminating hardcoded paths.

Programmatic Path Construction

Training scripts dynamically constructed dataset paths based on configuration values, ensuring consistent version usage.

Logging and Metadata Recording

Each training run logged dataset paths and version details to maintain audit trails.

Fail-Fast Validation

Training terminated immediately if the specified dataset version was missing.

Limitations Encountered

- Config updates were manual and error-prone.
- Data generation and config updates were not atomic operations.
- Dataset versions relied on labels rather than actual data validation.
- Accidental overwriting of dataset files could silently break reproducibility.

Conclusion

This approach worked in a single-developer environment but does not scale. Automated systems enforce dataset immutability and automatically track dataset lineage.

4. Based on this experience, what are the top three features you would want in an automated MLOps tool?

Based on project experience, the most essential features are:

1. Automatic Data Versioning

Manual version tracking is fragile and unreliable. Content-based versioning systems provide:

- Automatic dataset hashing

- Pipeline dependency tracking
- Guaranteed reproducibility
- Efficient storage management

2. Centralized Experiment Tracking

Manual metadata storage makes experiment comparison difficult. Automated experiment tracking provides:

- Automatic logging of parameters and metrics
- Visual experiment comparison dashboards
- Model artifact bundling
- Searchable experiment histories

3. Automated Monitoring and Alerting

Manual monitoring requires continuous human supervision. Automated monitoring provides:

- Continuous drift detection
- Performance degradation alerts
- Automated retraining triggers
- Production metric visualization dashboards

Conclusion

These three features address the core risks of manual MLOps: reproducibility failures, experiment tracking loss, and undetected production drift.

5. Describe the "breakdown" you experienced when trying to track which model version was currently serving requests in your API.

A major failure occurred when retraining overwrote a model file while the API was still serving an older in-memory version.

Failure Scenario

- Initial deployment loaded Model v1.
- Retraining replaced the model file.
- The API continued serving the old model.
- Deployment logs falsely indicated the latest model was active.

Investigation Findings

The system lacked:

- Deployment lifecycle tracking
- Model checksum verification
- File mutation detection
- Multi-instance deployment management
- Prediction logs linking predictions to model versions

Impact

This created uncertainty about which model generated predictions and undermined confidence in production outputs.

Conclusion

Manual deployment tracking provides limited visibility. Automated systems solve this using immutable model versions, deployment state tracking, and model integrity verification.

6. How would an automated Model Registry have made Phase B easier?

An automated model registry would significantly reduce manual effort and eliminate common deployment errors.

Current Manual Process

Phase B required:

- Manual model serialization
- Manual metadata creation
- Manual artifact tracking
- Manual verification procedures

This required significant development time and remained prone to errors.

Automated Model Registry Advantages

Automatic Metadata Capture

Automatically logs hyperparameters, metrics, environment details, and source code references.

Experiment Comparison

Provides dashboards to compare experiments, enabling informed hyperparameter tuning decisions.

Model Version Control

Ensures models are stored as immutable versions, enabling rollback and historical tracking.

Artifact Bundling

Packages models, preprocessing artifacts, configuration files, and evaluation outputs into a single reproducible unit.

Reproducibility Support

Allows entire training runs to be reproduced using stored experiment metadata.

Deployment Promotion Workflows

Supports controlled promotion from staging to production environments.

Conclusion

Automated model registries reduce development time, improve reliability, and enforce reproducibility standards. The manual approach demonstrated the complexity and error potential of managing model lifecycle processes without automation.

Overall Reflection

This project highlighted that manual MLOps pipelines are feasible for learning purposes but introduce significant scalability and reliability challenges. Automation is essential for maintaining reproducibility, ensuring deployment consistency, and supporting long-term production monitoring.